Task 1 — Advanced Theming Strategy: Material 3 + Brand Identity

Description:

In this task, students will explore how to apply and extend Material Design 3 (Material You) theming in their Compose app. They'll design a full theme system (colors, typography, shapes), integrate brand identity, support light/dark/dynamic modes, and document how this theming flows through their UI components. This is more than just "change colors" — they plan the theming architecture.

Practical Action Items:

1. **Research Theming Concepts:**
    - Write concise definitions (1–2 lines each) of:
        - colorScheme, typography, shapes in Material 3.
        - Dynamic colour (Android 12+), light/dark theme support.
    - Provide one short example snippet for each (e.g., how to declare a color scheme in Compose).
2. **Brand + Theme Design:**
    - Choose or design a "brand" for your app (name, logo idea, brand-colours).
    - Create a **color palette**: primary, secondary, tertiary, background, surface, error — define their hex values for both light and dark mode.
    - Choose **typography tokens**: heading, subtitle, body, caption (with font-sizes/sp values and font families).
    - Define **shape tokens**: e.g., smallCorner=4 dp, mediumCorner=12 dp, largeCorner=24 dp (for cards, buttons)

MindMatrix

3. **Theme Architecture Document:**

   - Document how you'll structure theme files: e.g., Theme.kt, Color.kt, Typography.kt, Shapes.kt.
   - Show how you'll wrap your app UI with MaterialTheme(colorScheme = ..., typography = ..., shapes = ...) { … }.
   - Annotate how components will reference theme tokens instead of hard-coded values (keeping consistency).

4. **Theming Across UI Components:**

   - Choose **three UI components** from your app (for example: AppBar, CardItem, Button). For each, specify how theme tokens will apply: colour, typography, shape, elevation.
   - Write a short mapping: "AppBar uses colorScheme.primary, typography.h6, shape.largeCorner".

5. **Reflection:**

   - 4-5 lines: Why is a well-structured theming system important for scalability and maintenance? What risks come from inconsistent UI styles (e.g., multiple hard-coded colours)?
   - Also mention: how dynamic colour and light/dark theming improve user experience.

Submission Options:

- PDF or Word file named YourName_Task1_AdvancedTheming.pdf containing: research notes, brand theme design, architecture doc, component mappings, reflection.
- Optional: image of your brand logo or palette graphic (embedded or link).

Outcome:

Students will become capable of designing and implementing a robust theming system in Compose that aligns with brand identity, supports light/dark/dynamic modes, and uses Material 3 best-practices. This sets the stage for consistent UI across the app.

Task 2 — Animation Design & Planning in Compose

Description:

Students will explore animations in Compose (spring, content-size change, animate*AsState(), AnimatedVisibility) and plan a set of interactive animations for an app feature. They will document animation types, triggers, states, performance considerations, and integrate them into UI design. This builds on theming but adds motion/interaction complexity.

Practical Action Items:

1. **Research Animation APIs:**
   - Define the following terms (1–2 lines each):
     - animateFloatAsState, animateColorAsState, AnimatedVisibility, animateContentSize, Transition.
   - Provide a short example snippet for one of them.
2. **Select an App Feature for Animation:**
   - Choose a UI screen or component from your app (e.g., a “Favourite” card expanding, a BottomBar item highlight, or an image gallery transition).
   - Write 2–3 lines describing how animation will enhance user experience.
3. **Animation Plan Document:**
   - Create a table for at least **three animation scenarios** in your feature: | Scenario | Trigger (user action) | Animation type | UI elements affected | Duration/Spec |
   - For example: “Card expand” | Tap card header | animateContentSize() with spring | Card height, internal Text & Image | 300 ms spring with damping=0.5
4. **Performance & Accessibility Notes:**

- Document two performance considerations: e.g., avoid heavy animations on large lists, respect prefers-reduced-motion accessibility settings.
- Document two accessibility/motion reduction considerations: e.g., allow disable animation, ensure content remains visible/non-motion.

5. **Wireframe Sequence & Storyboard:**
   - Sketch or use digital tool to show before-state, during animation, after-state of one of your scenarios.
   - Annotate which composables and modifiers help enable the animation (e.g., animateFloatAsState, Modifier.graphicsLayer, Modifier.alpha).
6. **Reflection:**
   - 4–5 lines: How does designing animation before coding improve UX and maintainability? What are pitfalls of overusing animation? How will you decide when not to animate?

Submission Options:

- PDF/Word named YourName_Task2_AnimationDesign.pdf containing: research definitions, feature description, animation plan table, performance & accessibility notes, storyboard, reflection.
- Optional: link to short screen-record/gif mock of the animation if you prototype.

Outcome:

Students will be able to plan and document animations in their Compose app with clarity, integrating them into UI design while considering performance and accessibility constraints. They'll be ready to implement expressive, polished UI interactions.

Task 3 — Theme + Animation Integration: Building a Polished UI Module

Description:

In this task, students will design a full UI module (a screen or component) that integrates their theming system (from Task 1) and animations (from Task 2). They will produce a detailed blueprint for implementation: data model, UI composition, theme application, animation triggers, and state changes. This is complex and mirrors real craft-grade UI work.

Practical Action Items:

1. **Define the Module:**
   - Choose a screen or component such as *"Gallery of Cards with Expand/ Collapse and Favourite"*, *"Profile Screen with Animated Theme Switcher"*, *"Dashboard with Themed Tabs and Animated Entry"*.
   - Write down a short description (2–3 lines) of what the module does and why it matters visually and functionally.
2. **Data Model & State Variables:**
   - Define the data model (class) you'll use for items in the module (if applicable).
   - Define state variables related to animation and theming: e.g., isExpanded, isFavourite, currentThemeMode, animationProgress.
   - Map which theme tokens will drive visual states (e.g., favourite icon colour = colorScheme.secondary, expanded card background = colorScheme.surfaceVariant).

MindMatrix

3. **UI Composition Plan:**

- Sketch the layout: include top-level MaterialTheme wrapper, surface of module, list or grid of items (if applicable).
- Annotate which composables will be used, which modifiers, and how theme/animation integrate (e.g., Card(shape = shapes.medium, backgroundColor = colorScheme.surface), expanded content with animateContentSize()).
- Indicate animation triggers (e.g., tapping favourite, toggling theme switch) and their effects.

4. **Implementation Timeline and Dependencies:**

- Create a mini-timeline of tasks:
  - Step 1: Set up theme tokens and apply.
  - Step 2: Build static UI layout.
  - Step 3: Add animation triggers and logic.
  - Step 4: Test responsiveness, performance, theme switch, dark/light.
- List external libraries or dependencies (if any) you'll use: e.g., Coil for images, Accompanist for placeholder, etc.

5. **Quality, Accessibility & Performance Checklist:**

- Make a checklist covering: correct use of theme tokens, ensure motion reduction support, efficient list (if used), low overdraw, appropriate image sizes, theme switch smoothness.
- Provide two test cases for edge conditions: e.g., theme change while animation in progress; very slow device toggles favourite repeatedly.

6. **Reflection:**

   - 4–5 lines reflecting: What was your biggest challenge integrating theme + animation? How do these two aspects combine to improve user satisfaction? What would you caution future developers about?

Submission Options:

- PDF/Word named YourName_Task3_ModuleBlueprint.pdf with: module description, data/state plan, UI composition plan, timeline, checklist & test cases, reflection.
- Optional: link to wireframe/design tool file or interim implementation code.

Outcome:

Students will produce a high-quality blueprint for a Compose UI module that blends theming, animation, data/state, and performance considerations. They'll be ready to code a well-crafted feature with design fidelity, smooth transitions, brand consistency and responsiveness.

Task 4 — Capstone: Build, Document & Present Themed Animated Feature

Description:

In this final capstone task of the session, students will **build (or partially build)** the UI module they designed in Task 3, document their implementation choices, capture real screenshots or video, evaluate performance and theme consistency, and prepare a short presentation of their work and learnings. This is the most complex and integrative task yet.

Practical Action Items:

1. **Implementation Snapshot:**
   - Build the module (as per Task 3) in your Compose project. At minimum, static layout + theming + one animation scenario must work.
   - Capture two screenshots or a short video: one in light theme, one in dark theme; one before animation, one during/after.
2. **Implementation Documentation:**
   - In a doc, describe what you implemented: which theme tokens you used, how you applied animation, where you deviated from your blueprint and why.
   - Provide code snippets or references to your project (e.g., Theme.kt changes, composable file for module, animation logic).
   - Evaluate performance: Did animation cause jank? Did theme switching produce flicker? What improvements could you make?
3. **Presentation Slide Deck (min 3 slides):**
   - Slide 1: Feature overview and why it matters.
   - Slide 2: Theming + UI preview (screenshots).
   - Slide 3: Animation flow + lessons learned.
   - Optional Slide 4: Next steps / future enhancements.

4. **Reflection & Improvement Log:**

- 4–5 lines: What surprised you during implementation? How will you proceed when building future features with theme + animation? What would you do differently?
- List two “future enhancements” you’d like to add to your module (e.g., animate favourite count badge, add dynamic colour based on user wallpaper).

5. **Submission Checklist:**

- Document or PDF named YourName_Task4_CapstoneFeature.pdf containing: documentation, screenshots/video link, reflection.
- Slide deck file or link.
- Link to your GitHub repo (if available) with module code or branch.

Outcome:

Students will experience full lifecycle of a UI feature: design → theme → animation → implementation → evaluation → presentation. They’ll stand up a meaningful Compose module with brand-quality theming and dynamic behaviour, and learn how to document, present and reflect on their work.